
MicroJava Mini-Compiler Project

Comprehensive Architectural Flow, Component Linkage, and Implementation
Report

Authors

ESA Irfan (2023-CS-104)
Hamdan Aleem (2023-CS-107)

Department of Computer Science

University of Engineering and Technology (UET), Lahore

Instructor

Ms. Aaliya

Abstract

This report details the design and implementation of a multi-stage syntax validator and execution engine for the **MicroJava** language. The compiler incorporates five core phases: double-buffered lexical analysis, recursive descent parsing with dynamic ASCII tree visualization, chained-scope symbol table management, LL(1) non-recursive predictive parsing using an explicit stack, and Canonical LR(1) bottom-up parsing. We explore grammar transformations required to resolve left recursion and left factoring for LL(1) predictive parsing. We also justify the choice of Canonical LR(1) for bottom-up parsing and present end-to-end execution traces mapping input source code through every compilation layer.

Contents

1	Introduction and Language Overview	3
1.1	Introduction to MicroJava	3
1.2	Key Implementation Features	3
2	Compiler Project Code Files and Linkage	3
3	MicroJava EBNF Grammar Specification	4
3.1	Left Recursion and Left Factoring Solutions	4
3.1.1	Eliminating Left Recursion	4
3.1.2	Left Factoring	5
3.2	Full Transformed LL(1) Grammar	5
4	Theoretical Logic and Implementation Details	7
4.1	Lexical Analyzer (Lexer)	7
4.1.1	Theoretical Logic	7
4.2	Recursive Descent Parser & Tree Logic	7
4.2.1	Theoretical Logic	7
4.2.2	Tree Generation Logic	7
4.3	Non-Recursive LL(1) Stack Parser	7
4.3.1	Theoretical Logic	7
4.4	Canonical LR(1) Parser	7
4.4.1	Theoretical Logic and Choice Justification	7
4.4.2	Standard Shift-Reduce Algorithm	8
5	Grammar Parsing Sets and Representative Tables	9
5.1	FIRST Sets	9
5.2	FOLLOW Sets	10
5.3	LL(1) Predictive Parsing Table (Representative Subset)	11
6	Canonical LR(1) Action, GOTO, and Trace Output	12
6.1	Action and GOTO Tables	12
6.2	Trace Segment for Declaration Statement: <code>int x;</code>	27
7	Unified End-to-End Multiphase Execution Trace	28
7.1	Example MicroJava Source Program	28
7.2	Phase 1: Lexical Analyzer Output Stream	29
7.3	Phase 3: Symbol Table Registry	31
7.4	Phase 4: LL(1) Parsing Rules (Actions Only)	31
7.5	Phase 5: Canonical LR(1) Actions (Actions Only)	33
8	Compiler Error Handling and Halt-on-Error Semantics	35
8.1	Error Handling Test Cases	35
9	Conclusion	37

1 Introduction and Language Overview

1.1 Introduction to MicroJava

MicroJava is a subset of Java designed for compiler construction courses. It is a strongly-typed, object-oriented language that supports basic data types (integers and characters), constants, arrays, static class-like structures, methods, nested scopes, and control flow statements such as conditional execution and loops.

1.2 Key Implementation Features

The compiler implementation incorporates the following architectural highlights:

- **C++17 Compiler Core:** Highly performant binary execution layer supporting five synchronized analysis phases.
- **Single-Error Termination:** All three parser engines stop execution immediately at the first lexical, syntax, or semantic error, preventing cascading failures.
- **Visual Representation:** The Recursive Descent parser outputs a colored ASCII concrete syntax tree illustrating exact scope nesting and grammar expansions.

2 Compiler Project Code Files and Linkage

The project consists of multiple header files, C++ source modules, Python scripts, and frontend assets. They are linked together logically to form the compiler suite.

3 MicroJava EBNF Grammar Specification

The parser expects code written in accordance with the Context-Free Grammar (CFG) rules below:

```

1 Program      = "program" ident {ConstDecl | VarDecl | ClassDecl} "{" {
    MethodDecl} "}".
2 ConstDecl   = "final" Type ident "=" (number | charConst) ";".
3 VarDecl     = Type ident {"," ident} ";".
4 ClassDecl   = "class" ident "{" {VarDecl} "}".
5 MethodDecl  = (Type | "void") ident "(" [FormPars] ")" {VarDecl}
    Block.
6 FormPars    = Type ident {"," Type ident}.
7 Type        = ident "[" "[" "]" ".
8 Block       = "{" {Statement} "}".
9 Statement   = Designator ("=" Expr | ActPars) ";"
10             | "if" "(" Condition ")" Statement ["else" Statement]
11             | "while" "(" Condition ")" Statement
12             | "return" [Expr] ";"
13             | "read" "(" Designator ")" ";"
14             | "print" "(" Expr ["," number] ")" ";"
15             | Block
16             | ";".
17 ActPars     = "(" [Expr {"," Expr}] ".
18 Condition   = Expr Relop Expr.
19 Relop       = "==" | "!=" | ">" | ">=" | "<" | "<=".
20 Expr        = ["-"] Term {Addop Term}.
21 Addop       = "+" | "-".
22 Term        = Factor {Mulop Factor}.
23 Mulop       = "*" | "/" | "%".
24 Factor      = Designator [ActPars]
25             | number
26             | charConst
27             | "new" ident "[" Expr "]"
28             | "(" Expr ")".
29 Designator  = ident {"." ident | "[" Expr "]"}.

```

Listing 1: MicroJava EBNF Grammar

3.1 Left Recursion and Left Factoring Solutions

To build a top-down parser such as LL(1) or Recursive Descent, the grammar must be free from:

1. **Left Recursion:** Rules of the form $A \rightarrow A\alpha \mid \beta$ cause infinite recursion in top-down parsers.
2. **Common Prefixes:** Rules of the form $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ prevent deterministic single-token lookahead.

3.1.1 Eliminating Left Recursion

Direct left recursion is eliminated using the standard transformation:

General Rule:

$$A \rightarrow A\alpha \mid \beta \quad \Longrightarrow \quad A \rightarrow \beta A' \quad A' \rightarrow \alpha A' \mid \varepsilon$$

Example Expression Grammar:

Original (left-recursive):

$$Expr \rightarrow Expr \text{ Addop } Term \mid Term$$

Transformed:

$$Expr \rightarrow Term Expr' \quad Expr' \rightarrow \text{Addop } Term Expr' \mid \varepsilon$$

3.1.2 Left Factoring

Common prefixes are factored out to allow deterministic parsing:

General Rule:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \quad \Longrightarrow \quad A \rightarrow \alpha A' \quad A' \rightarrow \beta_1 \mid \beta_2$$

Example Designator Tail:

Original:

$$Designator \rightarrow ident.ident \mid ident[Expr] \mid ident$$

Transformed:

$$Designator \rightarrow ident Designator_Tail$$

$$Designator_Tail \rightarrow .ident Designator_Tail \mid [Expr] Designator_Tail \mid \varepsilon$$

3.2 Full Transformed LL(1) Grammar

Below is the complete, transformed MicroJava grammar after removing all left recursions and applying left factoring:

```

1 Program = "program" ident Program_Body "{" MethodDecls
   "}".
2 Program_Body = {ConstDecl | VarDecl | ClassDecl}.
3 ConstDecl = "final" Type ident "=" ConstValue ";".
4 ConstValue = number | charConst.
5 VarDecl = Type ident VarDecl_Tail ";".
6 VarDecl_Tail = {"", " ident}.
7 ClassDecl = "class" ident "{" VarDecls "}".
8 VarDecls = {VarDecl}.
9 MethodDecls = {MethodDecl}.
10 MethodDecl = MethodType ident "(" FormPars_Opt ")"
   VarDecls Block.
11 MethodType = Type | "void".
12 FormPars_Opt = [FormPars].
13 FormPars = Type ident FormPars_Tail.
14 FormPars_Tail = {"", " Type ident}.
15 Type = ident Type_Opt.
16 Type_Opt = ["[" "]" ].
17 Block = "{" Statements "}".
18 Statements = {Statement}.
19 Statement = Designator Statement_AssignOrCall ";"
```

```

20 | "if" "(" Condition ")" Statement Else_Opt
21 | "while" "(" Condition ")" Statement
22 | "return" Expr_Opt ";"
23 | "read" "(" Designator ")" ";"
24 | "print" "(" Expr Print_Opt ")" ";"
25 | Block
26 | ";".
27 Statement_AssignOrCall = "=" Expr | ActPars.
28 Else_Opt                = ["else" Statement].
29 Expr_Opt                = [Expr].
30 Print_Opt               = [",", number].
31 ActPars                 = "(" ActPars_List ")".
32 ActPars_List            = [Expr ActPars_Tail].
33 ActPars_Tail            = {",", Expr}.
34 Condition               = Expr Relop Expr.
35 Relop                   = "==" | "!=" | ">" | ">=" | "<" | "<=".
36 Expr                    = Expr_Minus Term Expr_Tail.
37 Expr_Minus              = ["-"].
38 Expr_Tail               = {Addop Term}.
39 Addop                   = "+" | "-".
40 Term                    = Factor Term_Tail.
41 Term_Tail               = {Mulop Factor}.
42 Mulop                   = "*" | "/" | "%".
43 Factor                  = Designator Factor_Designator
44 | number
45 | charConst
46 | "new" ident Factor_New
47 | "(" Expr ")".
48 Factor_Designator       = [ActPars].
49 Factor_New              = ["[" Expr "]["].
50 Designator              = ident Designator_Tail.
51 Designator_Tail         = {".", ident | "[" Expr "]"}.

```

Listing 2: Full Transformed LL(1) Grammar

4 Theoretical Logic and Implementation Details

4.1 Lexical Analyzer (Lexer)

4.1.1 Theoretical Logic

The lexer converts a sequence of characters into a sequence of tokens, reading from a file stream and partitioning them based on regular definitions of the vocabulary:

- *Keyword* = **program** | **class** | **void** | **final** | **new** | **if** | **else** | **while** | **read** | **print** | **return**
- *Identifier* = letter (letter | digit)*
- *Number* = digit⁺
- *CharConst* = ' any__printable__char '

It uses a **double-buffering scheme** to process characters efficiently and tracks character coordinates (line and column numbers) to pinpoint lexical errors.

4.2 Recursive Descent Parser & Tree Logic

4.2.1 Theoretical Logic

The Recursive Descent (RD) parser is a top-down validation system that matches the token sequence by recursively calling parsing functions corresponding to each non-terminal.

4.2.2 Tree Generation Logic

The parser outputs a text-based Concrete Syntax Tree (CST) by tracking the call stack depth via a nesting counter:

- When a parsing function (e.g., `parseVarDecl()`) is entered, `indentLevel` is incremented.
- A branch node name is printed prefixed by vertical lines (`indentLevel × |`).
- When the function returns, `indentLevel` is decremented.

Because this tree corresponds exactly to the parser's active execution stack, it represents the leftmost derivation hierarchy of the input program.

4.3 Non-Recursive LL(1) Stack Parser

4.3.1 Theoretical Logic

The LL(1) parser uses a non-recursive predictive parsing model with an explicit stack. It looks up transitions in the 2-D parser table $M[A, a]$:

- If the top of the stack is a terminal matching the lookahead token, it pops and advances.
- If the top is a non-terminal A , it queries $M[A, a]$ to retrieve rule $A \rightarrow Y_1 Y_2 \dots Y_k$, pops A , and pushes $Y_k, Y_{k-1}, \dots, Y_1$ onto the stack in reverse order.

4.4 Canonical LR(1) Parser

4.4.1 Theoretical Logic and Choice Justification

The LR(1) parser is a bottom-up shift-reduce syntax validator. **Canonical LR(1)** was selected over LR(0), SLR(1), or LALR(1) for the following reasons:

1. **Explicit Lookahead Handling:** LR(1) embeds lookahead items $[A \rightarrow \alpha \cdot \beta, a]$ directly within states, resolving shift/reduce conflicts that SLR(1) cannot handle via FOLLOW sets alone.
2. **No Conflict Merging:** Unlike LALR(1), which merges states with the same core items but different lookaheads, Canonical LR(1) keeps these states split, guaranteeing strict parsing correctness.
3. **Handles Complex Subsets:** MicroJava contains nested expression boundaries and variable scopes where shift-reduce decisions depend heavily on the lookahead; LR(1) guarantees unambiguous transitions.

4.4.2 Standard Shift-Reduce Algorithm

The parser maintains a stack of states $s_0 s_1 \dots s_m$ and a corresponding symbol stack $X_1 X_2 \dots X_m$. For each step, let s be the top state and a the current lookahead. It queries ACTION[s, a]:

1. **Shift to state s' :** Push a onto the symbol stack, push s' onto the state stack, fetch the next token.
2. **Reduce by $A \rightarrow \beta$:** Pop $r = |\beta|$ symbols and states. Let the new top state be s'' . Query $s_{\text{new}} = \text{GOTO}[s'', A]$; push A and s_{new} .
3. **Accept:** Terminate with success.
4. **Error:** Halt immediately; print line and column number.

5 Grammar Parsing Sets and Representative Tables

5.1 FIRST Sets

Non-Terminal	FIRST Set
ActPars	(
ActPars_List	ϵ , (, -, charConst, ident, new, number
ActPars_Tail	ϵ , ,
Addop	+, -
Block	{
ClassDecl	class
Condition	(, -, charConst, ident, new, number
ConstDecl	final
ConstValue	charConst, number
Designator	ident
Designator_Tail	ϵ , ., [
Else_Opt	ϵ , else
Expr	(, -, charConst, ident, new, number
Expr_Minus	ϵ , -
Expr_Opt	ϵ , (, -, charConst, ident, new, number
Expr_Tail	ϵ , +, -
Factor	(, charConst, ident, new, number
Factor_Designator	ϵ , (
Factor_New	ϵ , [
FormPars	ident
FormPars_Opt	ϵ , ident
FormPars_Tail	ϵ , ,
MethodDecl	ident, void
MethodDecls	ϵ , ident, void
MethodType	ident, void
Mulop	%, *, /
Print_Opt	ϵ , ,
Program	program
Program_Body	ϵ , class, final, ident
Relop	!=, <, <=, ==, >, >=
Statement	;, ident, if, print, read, return, while, {
Statement_AssignOrCa	(, =
Statements	ϵ , ;, ident, if, print, read, return, while, {
Term	(, charConst, ident, new, number
Term_Tail	ϵ , %, *, /
Type	ident
Type_Opt	ϵ , [
VarDecl	ident
VarDecl_Tail	ϵ , ,
VarDecls	ϵ , ident

5.2 FOLLOW Sets

Non-Terminal	FOLLOW Set
ActPars	!=, %,), *, +, ,, -, /, ;;, <, <=, ==, >, >=,]
ActPars_List	)
ActPars_Tail	)
Addop	(, charConst, ident, new, number
Block	;;, else, ident, if, print, read, return, void, while, {, }
ClassDecl	class, final, ident, {
Condition	)
ConstDecl	class, final, ident, {
ConstValue	;
Designator	!=, %, (,), *, +, ,, -, /, ;;, <, <=, =, ==, >, >=,]
Designator_Tail	!=, %, (,), *, +, ,, -, /, ;;, <, <=, =, ==, >, >=,]
Else_Opt	;;, else, ident, if, print, read, return, while, {, }
Expr	!=,), ,, ;;, <, <=, ==, >, >=,]
Expr_Minus	(, charConst, ident, new, number
Expr_Opt	;
Expr_Tail	!=,), ,, ;;, <, <=, ==, >, >=,]
Factor	!=, %,), *, +, ,, -, /, ;;, <, <=, ==, >, >=,]
Factor_Designator	!=, %,), *, +, ,, -, /, ;;, <, <=, ==, >, >=,]
Factor_New	!=, %,), *, +, ,, -, /, ;;, <, <=, ==, >, >=,]
FormPars	)
FormPars_Opt	)
FormPars_Tail	)
MethodDecl	ident, void, }
MethodDecls	}
MethodType	ident
Mulop	(, charConst, ident, new, number
Print_Opt	)
Program	\$
Program_Body	{
Relop	(, -, charConst, ident, new, number
Statement	;;, else, ident, if, print, read, return, while, {, }
Statement_AssignOrCa	;;, else, ident, if, print, read, return, while, {, }
Statements	}
Term	!=,), +, ,, -, ;;, <, <=, ==, >, >=,]
Term_Tail	!=,), +, ,, -, ;;, <, <=, ==, >, >=,]
Type	ident
Type_Opt	ident
VarDecl	class, final, ident, {, }
VarDecl_Tail	;
VarDecls	{, }

5.3 LL(1) Predictive Parsing Table (Representative Subset)

Non-Terminal	Input Token	Applied Production Rule
ActPars	(	$ActPars \rightarrow (ActPars_List)$
ActPars_List	(	$ActPars_List \rightarrow Expr ActPars_Tail$
ActPars_List	)	$ActPars_List \rightarrow \epsilon$
ActPars_List	-	$ActPars_List \rightarrow Expr ActPars_Tail$
ActPars_List	charConst	$ActPars_List \rightarrow Expr ActPars_Tail$
ActPars_List	ident	$ActPars_List \rightarrow Expr ActPars_Tail$
ActPars_List	new	$ActPars_List \rightarrow Expr ActPars_Tail$
ActPars_List	number	$ActPars_List \rightarrow Expr ActPars_Tail$
ActPars_Tail	)	$ActPars_Tail \rightarrow \epsilon$
ActPars_Tail	,	$ActPars_Tail \rightarrow , Expr ActPars_Tail$
Addop	+	$Addop \rightarrow +$
Addop	-	$Addop \rightarrow -$
Block	{	$Block \rightarrow \{ Statements \}$
ClassDecl	class	$ClassDecl \rightarrow \text{class } ident \{ VarDecls \}$
Condition	(	$Condition \rightarrow Expr Relop Expr$
Condition	-	$Condition \rightarrow Expr Relop Expr$
Condition	charConst	$Condition \rightarrow Expr Relop Expr$
Condition	ident	$Condition \rightarrow Expr Relop Expr$
Condition	new	$Condition \rightarrow Expr Relop Expr$
Condition	number	$Condition \rightarrow Expr Relop Expr$
ConstDecl	final	$ConstDecl \rightarrow \text{final } Type ident = ConstValue ;$
ConstValue	charConst	$ConstValue \rightarrow \text{charConst}$
ConstValue	number	$ConstValue \rightarrow \text{number}$
Designator	ident	$Designator \rightarrow ident Designator_Tail$

6 Canonical LR(1) Action, GOTO, and Trace Output

The Canonical LR(1) parser generates **381 states**. Below is the representative subset of the generated Action and GOTO tables for all 381 states, followed by an execution stack trace.

6.1 Action and GOTO Tables

State	Action Transitions (Terminal → Action)	GOTO Transitions (Non-Terminal → State)
0	program → S1	Program → 2
1	ident → S3	None
2	\$ → acc	None
3	final → S9, class → S10, ident → S6, { → R5	ClassDecl → 4, Program_Body → 5, VarDecl → 7, Type → 8, ConstDecl → 11
4	final → S9, class → S10, ident → S6, { → R5	ClassDecl → 4, Program_Body → 12, VarDecl → 7, Type → 8, ConstDecl → 11
5	{ → S13	None
6	[→ S15, ident → R27	Type_Opt → 14
7	final → S9, class → S10, ident → S6, { → R5	ClassDecl → 4, Program_Body → 16, VarDecl → 7, Type → 8, ConstDecl → 11
8	ident → S17	None
9	ident → S6	Type → 18
10	ident → S19	None
11	final → S9, class → S10, ident → S6, { → R5	ClassDecl → 4, Program_Body → 20, VarDecl → 7, Type → 8, ConstDecl → 11
12	{ → R4	None
13	ident → S6, void → S21, } → R16	MethodDecl → 22, Type → 23, MethodType → 24, MethodDecls → 25
14	ident → R25	None
15	] → S26	None
16	{ → R3	None
17	, → S28, ; → R11	VarDecl_Tail → 27
18	ident → S29	None
19	{ → S30	None
20	{ → R2	None
21	ident → R19	None
22	ident → S6, void → S21, } → R16	MethodDecl → 22, Type → 23, MethodType → 24, MethodDecls → 31
23	ident → R18	None
24	ident → S32	None
25	} → S33	None

State	Action Transitions	GOTO Transitions
26	ident → R26	None
27	; → S34	None
28	ident → S35	None
29	= → S36	None
30	} → R14, ident → S6	VarDecl → 37, Type → 38, VarDecls → 39
31	} → R15	None
32	(→ S40	None
33	\$ → R1	None
34	{ → R9, ident → R9, final → R9, class → R9	None
35	, → S28, ; → R11	VarDecl_Tail → 41
36	number → S42, charConst → S43	ConstValue → 44
37	} → R14, ident → S6	VarDecl → 37, Type → 38, VarDecls → 45
38	ident → S46	None
39	} → S47	None
40	) → R21, ident → S6	FormPars_Opt → 48, Type → 49, FormPars → 50
41	; → R10	None
42	; → R7	None
43	; → R8	None
44	; → S51	None
45	} → R13	None
46	; → R11, , → S28	VarDecl_Tail → 52
47	class → R12, { → R12, ident → R12, final → R12	None
48	) → S53	None
49	ident → S54	None
50	) → R20	None
51	class → R6, { → R6, ident → R6, final → R6	None
52	; → S55	None
53	ident → S6, { → R14	VarDecl → 56, Type → 57, VarDecls → 58
54	) → R24, , → S59	FormPars_Tail → 60
55	ident → R9, } → R9	None
56	{ → R14, ident → S6	VarDecl → 56, Type → 57, VarDecls → 61
57	ident → S62	None
58	{ → S64	Block → 63
59	ident → S6	Type → 65
60	) → R22	None
61	{ → R13	None
62	, → S28, ; → R11	VarDecl_Tail → 66
63	ident → R17, } → R17, void → R17	None

State	Action Transitions	GOTO Transitions
64	read → S76, print → S77, if → S78, while → S74, { → S68, ; → S67, return → S70, } → R30, ident → S73	Designator → 69, Statements → 71, Statement → 72, Block → 75
65	ident → S79	None
66	; → S80	None
67	read → R38, } → R38, { → R38, ident → R38, while → R38, ; → R38, return → R38, if → R38, print → R38	None
68	read → S76, print → S77, if → S78, while → S74, { → S68, ; → S67, return → S70, } → R30, ident → S73	Designator → 69, Statements → 81, Statement → 72, Block → 75
69	= → S84, (→ S83	Statement_AssignOrCall → 82, ActPars → 85
70	- → S88, charConst → R61, ; → R44, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 86, Expr → 87, Expr_Opt → 89
71	} → S90	None
72	read → S76, print → S77, if → S78, while → S74, { → S68, ; → S67, return → S70, } → R30, ident → S73	Designator → 69, Statements → 91, Statement → 72, Block → 75
73	(→ R84, = → R84, [→ S92, . → S94	Designator_Tail → 93
74	(→ S95	None
75	{ → R37, ; → R37, while → R37, ident → R37, print → R37, return → R37, } → R37, if → R37, read → R37	None
76	(→ S96	None
77	(→ S97	None
78	(→ S98	None
79	) → R24, , → S59	FormPars_Tail → 99
80	{ → R9, ident → R9	None
81	} → S100	None
82	; → R31, while → R31, ident → R31, return → R31, if → R31, } → R31, print → R31, read → R31, { → R31	None
83	- → S88, number → R61, charConst → R61, ident → R61, (→ R61, new → R61,) → R49	Expr_Minus → 101, ActPars_List → 102, Expr → 103
84	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 86, Expr → 104
85	; → S105	None
86	(→ S108, new → S113, charConst → S109, ident → S111, number → S107	Term → 106, Factor → 110, Designator → 112
87	; → R43	None
88	number → R60, charConst → R60, (→ R60, ident → R60, new → R60	None
89	; → S114	None
90	} → R28, ident → R28, void → R28	None

State	Action Transitions	GOTO Transitions
91	} → R29	None
92	- → S88, number → R61, charConst → R61, ident → R61, (→ R61, new → R61	Expr_Minus → 115, Expr → 116
93	= → R81, (→ R81	None
94	ident → S117	None
95	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 118, Condition → 119, Expr → 120
96	ident → S122	Designator → 121
97	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 101, Expr → 123
98	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 118, Condition → 124, Expr → 120
99	) → R23	None
100	{ → R28, ; → R28, ident → R28, while → R28, print → R28, return → R28, if → R28, } → R28, read → R28	None
101	(→ S127, new → S132, charConst → S128, number → S125, ident → S130	Term → 126, Factor → 129, Designator → 131
102	) → S133	None
103	) → R51, , → S134	ActPars_Tail → 135
104	; → S136	None
105	if → R40, print → R40, } → R40, read → R40, { → R40, ident → R40, while → R40, ; → R40, return → R40	None
106	- → S138, + → S137, ; → R63	Addop → 139, Expr_Tail → 140
107	; → R73, * → R73, + → R73, / → R73, - → R73, % → R73	None
108	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 141, Expr → 142
109	* → R74, + → R74, % → R74, - → R74, ; → R74, / → R74	None
110	* → S143, / → S144, % → S145, ; → R68, + → R68, - → R68	Term_Tail → 146, Mulop → 147
111	[→ S148, . → S150, * → R84, ; → R84, (→ R84, - → R84, + → R84, / → R84, % → R84	Designator_Tail → 149
112	* → R78, (→ S152, ; → R78, + → R78, / → R78, % → R78, - → R78	Factor_Designator → 151, ActPars → 153
113	ident → S154	None
114	print → R34, return → R34, } → R34, if → R34, read → R34, ident → R34, { → R34, ; → R34, while → R34	None
115	(→ S157, new → S162, charConst → S158, ident → S160, number → S155	Term → 156, Factor → 159, Designator → 161
116	] → S163	None
117	(→ R84, = → R84, [→ S92, . → S94	Designator_Tail → 164

State	Action Transitions	GOTO Transitions
118	new → S172, ident → S170, (→ S167, charConst → S168, number → S165	Term → 166, Factor → 169, Designator → 171
119	) → S173	None
120	== → S176, != → S177, > → S178, >= → S179, < → S180, <= → S174	Relop → 175
121	) → S181	None
122	. → S182,) → R84, [→ S183	Designator_Tail → 184
123	, → S185,) → R46	Print_Opt → 186
124	) → S187	None
125	) → R73, * → R73, + → R73, / → R73, - → R73, , → R73, % → R73	None
126	- → S138,) → R63, + → S137, , → R63	Addop → 188, Expr_Tail → 189
127	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 141, Expr → 190
128	* → R74, + → R74, % → R74, - → R74,) → R74, / → R74, , → R74	None
129	* → S143, / → S144, % → S145, , → R68, + → R68, - → R68,) → R68	Term_Tail → 191, Mulop → 192
130	) → R84, [→ S194, * → R84, . → S193, (→ R84, - → R84, , → R84, + → R84, / → R84, % → R84	Designator_Tail → 195
131	* → R78, (→ S197, + → R78, / → R78, % → R78, , → R78, - → R78,) → R78	Factor_Designator → 196, ActPars → 198
132	ident → S199	None
133	; → R47	None
134	- → S88, number → R61, charConst → R61, ident → R61, (→ R61, new → R61	Expr_Minus → 101, Expr → 200
135	) → R48	None
136	if → R39, print → R39, read → R39, { → R39, ident → R39, while → R39, ; → R39, } → R39, return → R39	None
137	number → R64, new → R64, ident → R64, (→ R64, charConst → R64	None
138	number → R65, ident → R65, new → R65, (→ R65, charConst → R65	None
139	(→ S108, new → S113, charConst → S109, ident → S111, number → S107	Term → 201, Factor → 110, Designator → 112
140	; → R59	None
141	(→ S204, new → S209, charConst → S205, ident → S207, number → S202	Term → 203, Factor → 206, Designator → 208
142	) → S210	None
143	ident → R69, (→ R69, charConst → R69, number → R69, new → R69	None
144	new → R70, (→ R70, ident → R70, charConst → R70, number → R70	None
145	ident → R71, (→ R71, new → R71, number → R71, charConst → R71	None

State	Action Transitions	GOTO Transitions
146	+ → R66, - → R66, ; → R66	None
147	(→ S108, new → S113, charConst → S109, ident → S111, number → S107	Factor → 211, Designator → 112
148	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 212
149	- → R81, (→ R81, % → R81, ; → R81, * → R81, + → R81, / → R81	None
150	ident → S213	None
151	; → R72, * → R72, + → R72, / → R72, % → R72, - → R72	None
152	- → S88, charConst → R61, (→ R61, new → R61,) → R49, number → R61, ident → R61	Expr_Minus → 101, ActPars_List → 214, Expr → 103
153	- → R77, ; → R77, * → R77, + → R77, / → R77, % → R77	None
154	[→ S215, * → R80, ; → R80, + → R80, / → R80, % → R80, - → R80	Factor_New → 216
155	* → R73, + → R73, / → R73, - → R73,] → R73, % → R73	None
156	- → S138, + → S137,] → R63	Addop → 217, Expr_Tail → 218
157	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 141, Expr → 219
158	* → R74, + → R74,] → R74, % → R74, - → R74, / → R74	None
159	* → S143, / → S144, % → S145,] → R68, + → R68, - → R68	Term_Tail → 220, Mulop → 221
160	[→ S222, . → S224, * → R84, (→ R84, - → R84,] → R84, + → R84, / → R84, % → R84	Designator_Tail → 223
161	* → R78, (→ S226,] → R78, + → R78, / → R78, % → R78, - → R78	Factor_Designator → 225, ActPars → 227
162	ident → S228	None
163	(→ R84, = → R84, [→ S92, . → S94	Designator_Tail → 229
164	= → R82, (→ R82	None
165	>= → R73, == → R73, - → R73, > → R73, * → R73, + → R73, / → R73, <= → R73, < → R73, != → R73, % → R73	None
166	!= → R63, - → S138, + → S137, > → R63, <= → R63, < → R63, >= → R63, == → R63	Addop → 230, Expr_Tail → 231
167	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 141, Expr → 232
168	* → R74, + → R74, < → R74, / → R74, <= → R74, != → R74, % → R74, >= → R74, - → R74, > → R74, == → R74	None
169	* → S143, / → S144, <= → R68, >= → R68, == → R68, % → S145, - → R68, > → R68, < → R68, + → R68, != → R68	Term_Tail → 233, Mulop → 234

State	Action Transitions	GOTO Transitions
170	[→ S236, * → R84, . → S235, <= → R84, >= → R84, == → R84, (→ R84, - → R84, > → R84, < → R84, + → R84, / → R84, != → R84, % → R84	Designator_Tail → 237
171	>= → R78, == → R78, (→ S239, - → R78, > → R78, < → R78, * → R78, + → R78, / → R78, != → R78, % → R78, <= → R78	Factor_Designator → 238, ActPars → 240
172	ident → S241	None
173	read → S76, print → S77, if → S78, while → S74, { → S68, ; → S67, return → S70, ident → S73	Designator → 69, Block → 75, Statement → 242
174	number → R58, new → R58, ident → R58, (→ R58, charConst → R58, - → R58	None
175	- → S88, number → R61, charConst → R61, ident → R61, (→ R61, new → R61	Expr_Minus → 141, Expr → 243
176	ident → R53, new → R53, (→ R53, charConst → R53, - → R53, number → R53	None
177	(→ R54, ident → R54, new → R54, charConst → R54, - → R54, number → R54	None
178	new → R55, - → R55, ident → R55, (→ R55, charConst → R55, number → R55	None
179	new → R56, - → R56, ident → R56, (→ R56, number → R56, charConst → R56	None
180	ident → R57, (→ R57, charConst → R57, number → R57, new → R57, - → R57	None
181	; → S244	None
182	ident → S245	None
183	- → S88, number → R61, charConst → R61, ident → R61, (→ R61, new → R61	Expr_Minus → 115, Expr → 246
184	) → R81	None
185	number → S247	None
186	) → S248	None
187	read → S256, print → S257, if → S253, while → S254, { → S250, ; → S249, return → S252, ident → S73	Designator → 251, Block → 255, Statement → 258
188	(→ S127, new → S132, charConst → S128, number → S125, ident → S130	Term → 259, Factor → 129, Designator → 131
189	, → R59,) → R59	None
190	) → S260	None
191	+ → R66,) → R66, - → R66, , → R66	None
192	(→ S127, new → S132, charConst → S128, number → S125, ident → S130	Factor → 261, Designator → 131
193	ident → S262	None
194	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 263
195	- → R81, * → R81, + → R81, / → R81, , → R81, % → R81,) → R81, (→ R81	None

State	Action Transitions	GOTO Transitions
196	) → R72, * → R72, + → R72, / → R72, , → R72, % → R72, - → R72	None
197	- → S88, charConst → R61, (→ R61, new → R61,) → R49, number → R61, ident → R61	Expr_Minus → 101, ActPars_List → 264, Expr → 103
198	- → R77,) → R77, * → R77, + → R77, / → R77, , → R77, % → R77	None
199	[→ S265, * → R80, , → R80, + → R80, / → R80, % → R80, - → R80,) → R80	Factor_New → 266
200	) → R51, , → S134	ActPars_Tail → 267
201	- → S138, + → S137, ; → R63	Addop → 139, Expr_Tail → 268
202	) → R73, * → R73, + → R73, / → R73, - → R73, % → R73	None
203	- → S138, + → S137,) → R63	Addop → 269, Expr_Tail → 270
204	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 141, Expr → 271
205	* → R74, + → R74, % → R74, - → R74,) → R74, / → R74	None
206	* → S143, / → S144, % → S145, + → R68, - → R68,) → R68	Term_Tail → 272, Mulop → 273
207	) → R84, [→ S274, . → S276, * → R84, (→ R84, - → R84, + → R84, / → R84, % → R84	Designator_Tail → 275
208	* → R78, (→ S278, + → R78, / → R78, % → R78, - → R78,) → R78	Factor_Designator → 277, ActPars → 279
209	ident → S280	None
210	% → R76, / → R76, ; → R76, * → R76, + → R76, - → R76	None
211	* → S143, / → S144, % → S145, ; → R68, + → R68, - → R68	Term_Tail → 281, Mulop → 147
212	] → S282	None
213	[→ S148, . → S150, * → R84, ; → R84, (→ R84, + → R84, / → R84, % → R84, - → R84	Designator_Tail → 283
214	) → S284	None
215	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 285
216	- → R75, ; → R75, * → R75, + → R75, / → R75, % → R75	None
217	(→ S157, new → S162, charConst → S158, ident → S160, number → S155	Term → 286, Factor → 159, Designator → 161
218	] → R59	None
219	) → S287	None
220	] → R66, + → R66, - → R66	None
221	(→ S157, new → S162, charConst → S158, ident → S160, number → S155	Factor → 288, Designator → 161

State	Action Transitions	GOTO Transitions
222	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 289
223	- → R81, + → R81, (→ R81, % → R81, * → R81,] → R81, / → R81	None
224	ident → S290	None
225	* → R72,] → R72, + → R72, / → R72, % → R72, - → R72	None
226	- → S88, charConst → R61, (→ R61, new → R61,) → R49, number → R61, ident → R61	Expr_Minus → 101, ActPars_List → 291, Expr → 103
227	- → R77, * → R77, + → R77,] → R77, / → R77, % → R77	None
228	[→ S292, * → R80,] → R80, + → R80, / → R80, % → R80, - → R80	Factor_New → 293
229	(→ R83, = → R83	None
230	new → S172, ident → S170, (→ S167, charConst → S168, number → S165	Term → 294, Factor → 169, Designator → 171
231	<= → R59, >= → R59, != → R59, < → R59, > → R59, == → R59	None
232	) → S295	None
233	- → R66, != → R66, > → R66, == → R66, + → R66, < → R66, <= → R66, >= → R66	None
234	new → S172, ident → S170, (→ S167, charConst → S168, number → S165	Factor → 296, Designator → 171
235	ident → S297	None
236	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 298
237	- → R81, > → R81, == → R81, * → R81, + → R81, / → R81, < → R81, % → R81, <= → R81, >= → R81, != → R81, (→ R81	None
238	> → R72, == → R72, < → R72, * → R72, + → R72, / → R72, <= → R72, >= → R72, != → R72, % → R72, - → R72	None
239	- → S88, charConst → R61, (→ R61, new → R61,) → R49, number → R61, ident → R61	Expr_Minus → 101, ActPars_List → 299, Expr → 103
240	< → R77, * → R77, + → R77, / → R77, <= → R77, != → R77, % → R77, - → R77, >= → R77, > → R77, == → R77	None
241	* → R80, [→ S300, <= → R80, >= → R80, == → R80, - → R80, > → R80, < → R80, + → R80, / → R80, != → R80, % → R80	Factor_New → 301
242	if → R33, print → R33, } → R33, ; → R33, read → R33, { → R33, ident → R33, while → R33, return → R33	None
243	) → R52	None

State	Action Transitions	GOTO Transitions
244	read → R35, { → R35, while → R35, ident → R35, ; → R35, print → R35, return → R35, if → R35, } → R35	None
245	) → R84, . → S182, [→ S183	Designator_Tail → 302
246	] → S303	None
247	) → R45	None
248	; → S304	None
249	read → R38, } → R38, { → R38, return → R38, ident → R38, ; → R38, while → R38, else → R38, if → R38, print → R38	None
250	read → S76, print → S77, if → S78, while → S74, { → S68, ; → S67, return → S70, } → R30, ident → S73	Designator → 69, Statements → 305, Statement → 72, Block → 75
251	= → S307, (→ S83	Statement_AssignOrCall → 306, ActPars → 308
252	- → S88, charConst → R61, ; → R44, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 86, Expr → 87, Expr_Opt → 309
253	(→ S310	None
254	(→ S311	None
255	{ → R37, ; → R37, while → R37, ident → R37, else → R37, print → R37, return → R37, } → R37, if → R37, read → R37	None
256	(→ S312	None
257	(→ S313	None
258	return → R42, else → S315, if → R42, read → R42, { → R42, ident → R42, print → R42, while → R42, ; → R42, } → R42	Else_Opt → 314
259	- → S138,) → R63, + → S137, , → R63	Addop → 188, Expr_Tail → 316
260	) → R76, % → R76, / → R76, , → R76, * → R76, + → R76, - → R76	None
261	* → S143, / → S144, % → S145, , → R68, + → R68, - → R68,) → R68	Term_Tail → 317, Mulop → 192
262	) → R84, [→ S194, * → R84, . → S193, (→ R84, - → R84, , → R84, + → R84, / → R84, % → R84	Designator_Tail → 318
263	] → S319	None
264	) → S320	None
265	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 321
266	- → R75,) → R75, * → R75, + → R75, / → R75, , → R75, % → R75	None
267	) → R50	None
268	; → R62	None
269	(→ S204, new → S209, charConst → S205, ident → S207, number → S202	Term → 322, Factor → 206, Designator → 208
270	) → R59	None

State	Action Transitions	GOTO Transitions
271	) → S323	None
272	+ → R66, - → R66,) → R66	None
273	(→ S204, new → S209, charConst → S205, ident → S207, number → S202	Factor → 324, Designator → 208
274	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 325
275	- → R81, (→ R81, % → R81,) → R81, * → R81, + → R81, / → R81	None
276	ident → S326	None
277	) → R72, * → R72, + → R72, / → R72, % → R72, - → R72	None
278	- → S88, charConst → R61, (→ R61, new → R61,) → R49, number → R61, ident → R61	Expr_Minus → 101, ActPars_List → 327, Expr → 103
279	- → R77,) → R77, * → R77, + → R77, / → R77, % → R77	None
280	[→ S328, * → R80, + → R80, / → R80, % → R80, - → R80,) → R80	Factor_New → 329
281	- → R67, ; → R67, + → R67	None
282	[→ S148, . → S150, * → R84, ; → R84, (→ R84, - → R84, + → R84, / → R84, % → R84	Designator_Tail → 330
283	/ → R82, % → R82, + → R82, * → R82, - → R82, ; → R82, (→ R82	None
284	; → R47, * → R47, + → R47, - → R47, % → R47, / → R47	None
285	] → S331	None
286	- → S138, + → S137,] → R63	Addop → 217, Expr_Tail → 332
287	% → R76, / → R76,] → R76, * → R76, + → R76, - → R76	None
288	* → S143, / → S144, % → S145,] → R68, + → R68, - → R68	Term_Tail → 333, Mulop → 221
289	] → S334	None
290	[→ S222, . → S224, * → R84, (→ R84,] → R84, + → R84, / → R84, % → R84, - → R84	Designator_Tail → 335
291	) → S336	None
292	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 337
293	- → R75, * → R75, + → R75,] → R75, / → R75, % → R75	None
294	!= → R63, - → S138, + → S137, > → R63, <= → R63, < → R63, >= → R63, == → R63	Addop → 230, Expr_Tail → 338
295	> → R76, == → R76, * → R76, + → R76, < → R76, % → R76, <= → R76, != → R76, / → R76, >= → R76, - → R76	None

State	Action Transitions	GOTO Transitions
296	* → S143, / → S144, <= → R68, >= → R68, == → R68, % → S145, - → R68, > → R68, < → R68, + → R68, != → R68	Term_Tail → 339, Mulop → 234
297	[→ S236, * → R84, != → R84, . → S235, <= → R84, >= → R84, (→ R84, == → R84, - → R84, > → R84, < → R84, + → R84, / → R84, % → R84	Designator_Tail → 340
298	] → S341	None
299	) → S342	None
300	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 343
301	>= → R75, - → R75, > → R75, == → R75, < → R75, * → R75, + → R75, / → R75, <= → R75, != → R75, % → R75	None
302	) → R82	None
303	) → R84, . → S182, [→ S183	Designator_Tail → 344
304	read → R36, { → R36, ident → R36, ; → R36, while → R36, print → R36, return → R36, } → R36, if → R36	None
305	} → S345	None
306	; → R31, while → R31, ident → R31, else → R31, return → R31, if → R31, } → R31, print → R31, read → R31, { → R31	None
307	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 86, Expr → 346
308	; → S347	None
309	; → S348	None
310	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 118, Condition → 349, Expr → 120
311	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 118, Condition → 350, Expr → 120
312	ident → S122	Designator → 351
313	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 101, Expr → 352
314	; → R32, ident → R32, while → R32, print → R32, return → R32, if → R32, } → R32, read → R32, { → R32	None
315	read → S76, print → S77, if → S78, while → S74, { → S68, ; → S67, return → S70, ident → S73	Designator → 69, Block → 75, Statement → 353
316	, → R62,) → R62	None
317	- → R67, , → R67,) → R67, + → R67	None
318	/ → R82, , → R82, % → R82, - → R82,) → R82, (→ R82, * → R82, + → R82	None
319	) → R84, [→ S194, * → R84, . → S193, (→ R84, - → R84, , → R84, + → R84, / → R84, % → R84	Designator_Tail → 354

State	Action Transitions	GOTO Transitions
320	* → R47, + → R47, - → R47,) → R47, % → R47, / → R47, , → R47	None
321	] → S355	None
322	- → S138, + → S137,) → R63	Addop → 269, Expr_Tail → 356
323	) → R76, % → R76, / → R76, * → R76, + → R76, - → R76	None
324	* → S143, / → S144, % → S145, + → R68, - → R68,) → R68	Term_Tail → 357, Mulop → 273
325	] → S358	None
326	) → R84, [→ S274, . → S276, * → R84, (→ R84, + → R84, / → R84, % → R84, - → R84	Designator_Tail → 359
327	) → S360	None
328	- → S88, charConst → R61, (→ R61, new → R61, number → R61, ident → R61	Expr_Minus → 115, Expr → 361
329	- → R75,) → R75, * → R75, + → R75, / → R75, % → R75	None
330	- → R83, % → R83, + → R83, ; → R83, (→ R83, * → R83, / → R83	None
331	- → R79, ; → R79, / → R79, % → R79, * → R79, + → R79	None
332	] → R62	None
333	- → R67,] → R67, + → R67	None
334	[→ S222, . → S224, * → R84, (→ R84, - → R84,] → R84, + → R84, / → R84, % → R84	Designator_Tail → 362
335	/ → R82, + → R82, % → R82,] → R82, * → R82, - → R82, (→ R82	None
336	* → R47, + → R47,] → R47, - → R47, % → R47, / → R47	None
337	] → S363	None
338	!= → R62, >= → R62, < → R62, > → R62, == → R62, <= → R62	None
339	< → R67, + → R67, <= → R67, != → R67, - → R67, >= → R67, > → R67, == → R67	None
340	/ → R82, <= → R82, != → R82, % → R82, >= → R82, - → R82, (→ R82, > → R82, == → R82, * → R82, + → R82, < → R82	None
341	[→ S236, * → R84, . → S235, <= → R84, >= → R84, (→ R84, == → R84, - → R84, > → R84, < → R84, + → R84, / → R84, != → R84, % → R84	Designator_Tail → 364
342	- → R47, != → R47, > → R47, == → R47, <= → R47, * → R47, + → R47, < → R47, % → R47, / → R47, >= → R47	None
343	] → S365	None
344	) → R83	None

State	Action Transitions	GOTO Transitions
345	{ → R28, else → R28, ; → R28, ident → R28, while → R28, print → R28, return → R28, if → R28, } → R28, read → R28	None
346	; → S366	None
347	if → R40, print → R40, } → R40, read → R40, { → R40, else → R40, ident → R40, while → R40, ; → R40, return → R40	None
348	else → R34, print → R34, return → R34, } → R34, if → R34, read → R34, ident → R34, { → R34, ; → R34, while → R34	None
349	) → S367	None
350	) → S368	None
351	) → S369	None
352	, → S185,) → R46	Print_Opt → 370
353	{ → R41, ident → R41, ; → R41, while → R41, print → R41, } → R41, return → R41, if → R41, read → R41	None
354	, → R83, % → R83,) → R83, (→ R83, - → R83, * → R83, + → R83, / → R83	None
355	- → R79,) → R79, / → R79, % → R79, , → R79, * → R79, + → R79	None
356	) → R62	None
357	- → R67,) → R67, + → R67	None
358	) → R84, [→ S274, . → S276, * → R84, (→ R84, - → R84, + → R84, / → R84, % → R84	Designator_Tail → 371
359	/ → R82, % → R82, + → R82, * → R82, - → R82,) → R82, (→ R82	None
360	* → R47, + → R47, - → R47,) → R47, % → R47, / → R47	None
361	] → S372	None
362	- → R83, % → R83, + → R83, * → R83, (→ R83,] → R83, / → R83	None
363	- → R79, / → R79, % → R79, * → R79, + → R79,] → R79	None
364	< → R83, % → R83, <= → R83, >= → R83, != → R83, (→ R83, - → R83, > → R83, == → R83, * → R83, + → R83, / → R83	None
365	< → R79, / → R79, <= → R79, % → R79, != → R79, >= → R79, - → R79, > → R79, == → R79, * → R79, + → R79	None
366	print → R39, if → R39, read → R39, { → R39, ident → R39, while → R39, ; → R39, else → R39, } → R39, return → R39	None
367	read → S256, print → S257, if → S253, while → S254, { → S250, ; → S249, return → S252, ident → S73	Designator → 251, Block → 255, Statement → 373

State	Action Transitions	GOTO Transitions
368	read → S256, print → S257, if → S253, while → S254, { → S250, ; → S249, return → S252, ident → S73	Designator → 251, Block → 255, Statement → 374
369	; → S375	None
370	) → S376	None
371	- → R83, % → R83, + → R83,) → R83, (→ R83, * → R83, / → R83	None
372	- → R79,) → R79, / → R79, % → R79, * → R79, + → R79	None
373	return → R42, else → S378, if → R42, } → R42, read → R42, { → R42, ident → R42, ; → R42, while → R42, print → R42	Else_Opt → 377
374	return → R33, if → R33, print → R33, } → R33, ; → R33, read → R33, { → R33, ident → R33, while → R33, else → R33	None
375	read → R35, { → R35, while → R35, else → R35, ; → R35, ident → R35, print → R35, return → R35, if → R35, } → R35	None
376	; → S379	None
377	else → R32, ident → R32, while → R32, ; → R32, print → R32, return → R32, if → R32, } → R32, read → R32, { → R32	None
378	read → S256, print → S257, if → S253, while → S254, { → S250, ; → S249, return → S252, ident → S73	Designator → 251, Block → 255, Statement → 380
379	read → R36, { → R36, ident → R36, ; → R36, while → R36, else → R36, print → R36, return → R36, } → R36, if → R36	None
380	{ → R41, ident → R41, ; → R41, while → R41, else → R41, print → R41, } → R41, return → R41, if → R41, read → R41	None

6.2 Trace Segment for Declaration Statement: `int x;`

Step-by-Step LR(1) Stack Trace `int x;`

- Step 1. State 0.** Top of state stack: 0. Lookahead: identifier `int`.
- Step 2. Shift.** Shift `int` $\Rightarrow$ transition to **State 10**.
- Step 3. State 10.** Top: 10. Lookahead: identifier `x`.
- Step 4. Reduce R25.** Reduce `int` $\Rightarrow$ *Type* using rule $Type \rightarrow ident$. Pop 1 state. New top: 0. $GOTO[0, Type] = 8 \Rightarrow$ push **State 8**.
- Step 5. State 8.** Top: 8. Lookahead: `x`.
- Step 6. Shift.** Shift identifier `x` $\Rightarrow$ transition to **State 17**.
- Step 7. State 17.** Top: 17. Lookahead: `;`.
- Step 8. Reduce R11.** $VarDecl_Tail \rightarrow \varepsilon$. Pop 0 states. $GOTO[17, VarDecl_Tail] = 27 \Rightarrow$ push **State 27**.
- Step 9. State 27.** Top: 27. Shift `;` $\Rightarrow$ **State 34**.
- Step 10. State 34. Reduce R9.** $VarDecl \rightarrow Type\ ident\ VarDecl_Tail\ ;$. Pop 4 states.

7 Unified End-to-End Multiphase Execution Trace

7.1 Example MicroJava Source Program

```
1 program SimpleSum
2   int x;
3   int y;
4   int sum;
5   {
6     void main() {
7       x = 5;
8       y = 10;
9       sum = x + y;
10      print(sum);
11    }
12  }
```

Listing 3: Example MicroJava Program SimpleSum.mj

7.2 Phase 1: Lexical Analyzer Output Stream

#	Token Type	Lexeme	Line	Column
1	KW_PROGRAM	program	1	1
2	IDENTIFIER	SimpleSum	1	9
3	IDENTIFIER	int	3	3
4	IDENTIFIER	x	3	7
5	SEMICOLON	;	3	8
6	IDENTIFIER	int	4	3
7	IDENTIFIER	y	4	7
8	SEMICOLON	;	4	8
9	IDENTIFIER	int	5	3
10	IDENTIFIER	sum	5	7
11	SEMICOLON	;	5	10
12	LBRACE	{	6	1
13	KW_VOID	void	7	3
14	IDENTIFIER	main	7	8
15	LPAREN	(	7	12
16	RPAREN	)	7	13
17	LBRACE	{	7	15
18	IDENTIFIER	x	8	5
19	ASSIGN	=	8	7
20	NUMBER	5	8	9
21	SEMICOLON	;	8	10
22	IDENTIFIER	y	9	5
23	ASSIGN	=	9	7
24	NUMBER	10	9	9
25	SEMICOLON	;	9	11
26	IDENTIFIER	sum	10	5
27	ASSIGN	=	10	9
28	IDENTIFIER	x	10	11
29	PLUS	+	10	13
30	IDENTIFIER	y	10	15
31	SEMICOLON	;	10	16
32	KW_PRINT	print	11	5
33	LPAREN	(	11	10
34	IDENTIFIER	sum	11	11
35	RPAREN	)	11	14
36	SEMICOLON	;	11	15
37	RBRACE	}	12	3
38	RBRACE	}	13	1
39	EOF	EOF	14	1

[SUCCESS] Lexical scan completed with **0** lexical errors. Total tokens: **39**

Phase 2: Recursive Descent Concrete Syntax Tree

Concrete Syntax Tree SimpleSum.mj

```

Program
├─ KW_PROGRAM: program
├─ IDENTIFIER: SimpleSum
├─ VarDecl: int x
│   └─ Type: int
│       └─ IDENTIFIER: int
│   └─ IDENTIFIER: x
│   └─ SEMICOLON: ;
├─ VarDecl: int y
│   └─ Type: int
│       └─ IDENTIFIER: int
│   └─ IDENTIFIER: y
│   └─ SEMICOLON: ;
├─ VarDecl: int sum
│   └─ Type: int
│       └─ IDENTIFIER: int
│   └─ IDENTIFIER: sum
│   └─ SEMICOLON: ;
├─ LBRACE: {
├─ MethodDecl: void main
│   └─ KW_VOID: void
│   └─ IDENTIFIER: main
│   └─ LPAREN: (
│   └─ RPAREN: )
│   └─ Block
│       └─ LBRACE: {
│           └─ Statement: x = 5
│               └─ Designator
│                   └─ IDENTIFIER: x
│               └─ ASSIGN: =
│               └─ Expr
│                   └─ Term
│                       └─ Factor
│                           └─ NUMBER: 5
│               └─ SEMICOLON: ;
│           └─ Statement: y = 10
│               └─ Designator
│                   └─ IDENTIFIER: y
│               └─ ASSIGN: =
│               └─ Expr
│                   └─ Term
│                       └─ Factor
│                           └─ NUMBER: 10
│               └─ SEMICOLON: ;
│           └─ Statement: sum = x + y
│               └─ Designator
│                   └─ IDENTIFIER: sum
│               └─ ASSIGN: =
│               └─ Expr
│                   └─ Term
│                       └─ Factor
│                           └─ Designator
│                               └─ IDENTIFIER: x
│                   └─ Addop
│                       └─ PLUS: +
│                   └─ Term
│                       └─ Factor
│                           └─ Designator
│                               └─ IDENTIFIER: y
│               └─ SEMICOLON: ;
│           └─ Statement: print(sum)
│               └─ KW_PRINT: print
│               └─ LPAREN: (
│               └─ Expr
│                   └─ Term
│                       └─ Factor
│                           └─ Designator
│                               └─ IDENTIFIER: sum
│               └─ RPAREN: )
│               └─ SEMICOLON: ;
│       └─ RBRACE: }
└─ RBRACE: }

```

7.3 Phase 3: Symbol Table Registry

Scope	Symbol Name	Kind	Data Type	Line
1	main	METHOD	void	7
1	sum	VAR	int	5
1	y	VAR	int	4
1	x	VAR	int	3
0	SimpleSum	CLASS	void	1
0	len	METHOD	int	0
0	ord	METHOD	int	0
0	chr	METHOD	char	0

7.4 Phase 4: LL(1) Parsing Rules (Actions Only)

```

Applied Rule  Program -> program ident Program_Body { MethodDecls }
Match token  'program'
Match token  'ident'
Applied Rule  Program_Body -> VarDecl Program_Body
Applied Rule  VarDecl -> Type ident VarDecl_Tail ;
Applied Rule  Type -> ident Type_Opt
Match token  'ident'
Applied Rule  Type_Opt -> epsilon
Match token  'ident'
Applied Rule  VarDecl_Tail -> epsilon
Match token  ';'
Applied Rule  Program_Body -> VarDecl Program_Body
Applied Rule  VarDecl -> Type ident VarDecl_Tail ;
Applied Rule  Type -> ident Type_Opt
Match token  'ident'
Applied Rule  Type_Opt -> epsilon
Match token  'ident'
Applied Rule  VarDecl_Tail -> epsilon
Match token  ';'
Applied Rule  Program_Body -> VarDecl Program_Body
Applied Rule  VarDecl -> Type ident VarDecl_Tail ;
Applied Rule  Type -> ident Type_Opt
Match token  'ident'
Applied Rule  Type_Opt -> epsilon
Match token  'ident'
Applied Rule  VarDecl_Tail -> epsilon
Match token  ';'
Applied Rule  Program_Body -> epsilon
Match token  '{'
Applied Rule  MethodDecls -> MethodDecl MethodDecls
Applied Rule  MethodDecl -> MethodType ident ( FormPars_Opt ) VarDecls Block
Applied Rule  MethodType -> void
Match token  'void'
Match token  'ident'
Match token  '('
Applied Rule  FormPars_Opt -> epsilon
Match token  ')'
Applied Rule  VarDecls -> epsilon
Applied Rule  Block -> { Statements }
Match token  '{'
Applied Rule  Statements -> Statement Statements
Applied Rule  Statement -> Designator Statement_AssignOrCall
Applied Rule  Designator -> ident Designator_Tail
Match token  'ident'
Applied Rule  Designator_Tail -> epsilon

```



```

Applied Rule Statement_AssignOrCall -> = Expr ;
Match token '='
Applied Rule Expr -> Expr_Minus Term Expr_Tail
Applied Rule Expr_Minus -> epsilon
Applied Rule Term -> Factor Term_Tail
Applied Rule Factor -> number
Match token 'number'
Applied Rule Term_Tail -> epsilon
Applied Rule Expr_Tail -> epsilon
Match token ';'
Applied Rule Statements -> Statement Statements
Applied Rule Statement -> Designator Statement_AssignOrCall
Applied Rule Designator -> ident Designator_Tail
Match token 'ident'
Applied Rule Designator_Tail -> epsilon
Applied Rule Statement_AssignOrCall -> = Expr ;
Match token '='
Applied Rule Expr -> Expr_Minus Term Expr_Tail
Applied Rule Expr_Minus -> epsilon
Applied Rule Term -> Factor Term_Tail
Applied Rule Factor -> number
Match token 'number'
Applied Rule Term_Tail -> epsilon
Applied Rule Expr_Tail -> epsilon
Match token ';'
Applied Rule Statements -> Statement Statements
Applied Rule Statement -> Designator Statement_AssignOrCall
Applied Rule Designator -> ident Designator_Tail
Match token 'ident'
Applied Rule Designator_Tail -> epsilon
Applied Rule Statement_AssignOrCall -> = Expr ;
Match token '='
Applied Rule Expr -> Expr_Minus Term Expr_Tail
Applied Rule Expr_Minus -> epsilon
Applied Rule Term -> Factor Term_Tail
Applied Rule Factor -> Designator Factor_Designator
Applied Rule Designator -> ident Designator_Tail
Match token 'ident'
Applied Rule Designator_Tail -> epsilon
Applied Rule Factor_Designator -> epsilon
Applied Rule Term_Tail -> epsilon
Applied Rule Expr_Tail -> Addop Term Expr_Tail
Applied Rule Addop -> +
Match token '+'
Applied Rule Term -> Factor Term_Tail
Applied Rule Factor -> Designator Factor_Designator
Applied Rule Designator -> ident Designator_Tail
Match token 'ident'
Applied Rule Designator_Tail -> epsilon
Applied Rule Factor_Designator -> epsilon
Applied Rule Term_Tail -> epsilon
Applied Rule Expr_Tail -> epsilon
Match token ';'
Applied Rule Statements -> Statement Statements
Applied Rule Statement -> print ( Expr Print_Opt ) ;
Match token 'print'
Match token '('
Applied Rule Expr -> Expr_Minus Term Expr_Tail
Applied Rule Expr_Minus -> epsilon
Applied Rule Term -> Factor Term_Tail
Applied Rule Factor -> Designator Factor_Designator

```

```

Applied Rule  Designator -> ident Designator_Tail
Match token   'ident'
Applied Rule  Designator_Tail -> epsilon
Applied Rule  Factor_Designator -> epsilon
Applied Rule  Term_Tail -> epsilon
Applied Rule  Expr_Tail -> epsilon
Applied Rule  Print_Opt -> epsilon
Match token   '),'
Match token   ';'
Applied Rule  Statements -> epsilon
Match token   '}'
Applied Rule  MethodDecls -> epsilon
Match token   '}'
Match token   '$'

```

[SUCCESS] LL(1) Parser completed with **0** errors.

7.5 Phase 5: Canonical LR(1) Actions (Actions Only)

```

Shift 1
Shift 3
Shift 6
Reduce (27) Type_Opt.      Pop 0 states. Push GOTO(6, Type_Opt)      = 14.
Reduce (25) Type.         Pop 2 states. Push GOTO(3, Type)          = 8.
Shift 17
Reduce (11) VarDecl_Tail. Pop 0 states. Push GOTO(17, VarDecl_Tail) = 27.
Shift 34
Reduce (9) VarDecl.       Pop 4 states. Push GOTO(3, VarDecl)       = 7.
Shift 6
Reduce (27) Type_Opt.     Pop 0 states. Push GOTO(6, Type_Opt)     = 14.
Reduce (25) Type.        Pop 2 states. Push GOTO(7, Type)          = 8.
Shift 17
Reduce (11) VarDecl_Tail. Pop 0 states. Push GOTO(17, VarDecl_Tail) = 27.
Shift 34
Reduce (9) VarDecl.       Pop 4 states. Push GOTO(7, VarDecl)       = 7.
Shift 6
Reduce (27) Type_Opt.     Pop 0 states. Push GOTO(6, Type_Opt)     = 14.
Reduce (25) Type.        Pop 2 states. Push GOTO(7, Type)          = 8.
Shift 17
Reduce (11) VarDecl_Tail. Pop 0 states. Push GOTO(17, VarDecl_Tail) = 27.
Shift 34
Reduce (9) VarDecl.       Pop 4 states. Push GOTO(7, VarDecl)       = 7.
Reduce (5) Program_Body. Pop 0 states. Push GOTO(7, Program_Body) = 16.
Reduce (3) Program_Body. Pop 2 states. Push GOTO(7, Program_Body) = 16.
Reduce (3) Program_Body. Pop 2 states. Push GOTO(7, Program_Body) = 16.
Reduce (3) Program_Body. Pop 2 states. Push GOTO(3, Program_Body) = 5.
Shift 13
Shift 21
Reduce (19) MethodType.   Pop 1 state.  Push GOTO(13, MethodType) = 24.
Shift 32
Shift 40
Reduce (21) FormPars_Opt. Pop 0 states. Push GOTO(40, FormPars_Opt) = 48.
Shift 53
Reduce (14) VarDecls.     Pop 0 states. Push GOTO(53, VarDecls)    = 58.
Shift 64
Shift 73
Reduce (84) Designator_Tail. Pop 0 states. Push GOTO(73, Designator_Tail) =
93.

```

```
Reduce (81) Designator. Pop 2 states. Push GOTO(64, Designator) = 69.
Shift 84
Reduce (61) Expr_Minus. Pop 0 states. Push GOTO(84, Expr_Minus) = 86.
Shift 107
Reduce (73) Factor. Pop 1 state. Push GOTO(86, Factor) = 110.
Reduce (68) Term_Tail. Pop 0 states. Push GOTO(110, Term_Tail) = 146.
Reduce (66) Term. Pop 2 states. Push GOTO(86, Term) = 106.
Reduce (63) Expr_Tail. Pop 0 states. Push GOTO(106, Expr_Tail) = 140.
Reduce (59) Expr. Pop 3 states. Push GOTO(84, Expr) = 104.
Shift 136
Reduce (39) Statement_AssignOrCall. Pop 3 states. Push GOTO(69,
    Statement_AssignOrCall) = 82.
Reduce (31) Statement. Pop 2 states. Push GOTO(64, Statement) = 72.
Accept
```

[SUCCESS] Canonical LR(1) Parser completed with **0** errors.

8 Compiler Error Handling and Halt-on-Error Semantics

The compiler implements strict validation rules where **compilation halts immediately at the first syntax or semantic error**. When the parser detects an invalid statement or mismatched token:

- It raises a `std::runtime_error` inside the `reportError()` function.
- This prints the error details (with line/column coordinates) and aborts the parser call stack.
- The console displays all output generated *prior* to the failure, giving a clear visual boundary between valid parsed structures and the error point.

8.1 Error Handling Test Cases

The following five test cases demonstrate compiler behavior upon encountering syntax and semantic violations.

Test Case 1 | Lexical Error Invalid Symbol

Source: `int x # 5;`

Behavior: The double-buffered scanner encounters the invalid character `#` and immediately reports:

```
[LEXICAL ERROR] Line 3, Col 9: Unexpected character '#'
```

Halt: Compiler execution aborts immediately, preventing any further parsing.

Test Case 2 | Syntactic Error Missing Semicolon

Source: `int x (followed by int y;)`

Behavior: The recursive descent and LL(1) parsers expect a semicolon to terminate the declaration.

```
[SYNTAX ERROR] Line 4, Col 3: Expected ';' but found 'int'
```

Halt: Parse stacks are cleaned up and compilation is terminated.

Test Case 3 | Syntactic Error Mismatched Parenthesis

Source: `while (x > 0 { x = x - 1; }`

Behavior: The parsers detect a missing closing parenthesis for the loop condition.

```
[SYNTAX ERROR] Line 12, Col 16: Expected ')' but found '{'
```

Halt: Compiler execution is halted at the mismatched boundary.

Test Case 4 | Semantic Error Duplicate Variable Declaration

Source: `int x; int x;`

Behavior: Both declarations parse successfully, but scope duplicate verification in the symbol table fails.

```
[SEMANTIC ERROR] Line 4, Col 7: Redclaration of identifier 'x' in scope
1
```

Halt: Symbol table insertion triggers an immediate semantic halt.

Test Case 5 | Semantic Error Undeclared Identifier

Source: `sum = x + z;` (where `z` was never declared)

Behavior: During identifier resolution in the factor evaluation rules, the symbol table lookup returns null.

```
[SEMANTIC ERROR] Line 10, Col 13: Identifier 'z' is undeclared in this
scope
```

Halt: The compiler halts immediately to prevent invalid type-check cascades.

9 Conclusion

This compiler playground combines a modular C++ validating core with a responsive sandbox. The integration of five compilation layers synchronized to halt immediately on syntax failures guarantees correct, deterministic evaluation of MicroJava source files, serving as an effective practical platform for syntax analysis and compiler construction education.